# Observer

Season: 3 · Behavioral · Category: GoF · Level: Intermediate · Build with Waqar

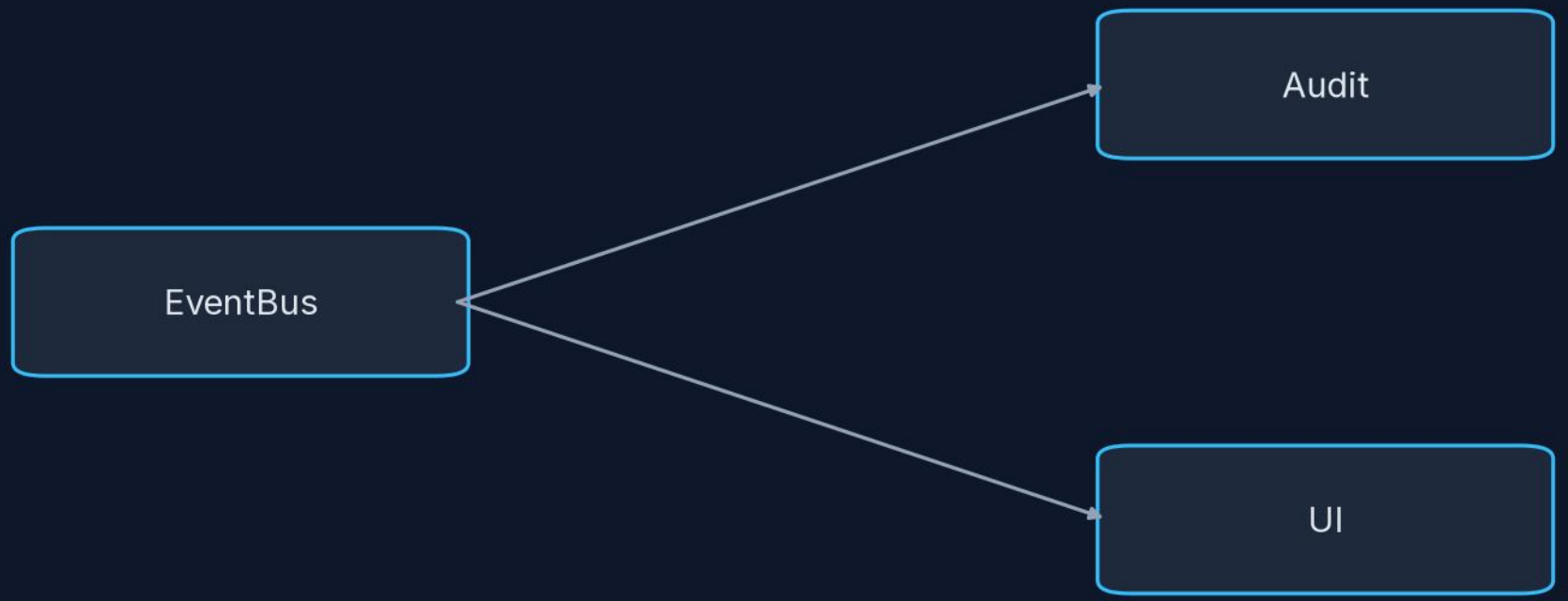
# Today's Hook

- Today: Observer as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

# The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

# Structure




Observer structure

# Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

# Naive Code

```
// Naive sketch for Observer
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

# Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

# Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/behavioral/observer/EventBus.java
package com.buildwithwaqar.patterns.gof.behavioral.observer;

import java.util.ArrayList;
import java.util.List;
import java.util.function.Consumer;

public final class EventBus<T> {
    private final List<Consumer<T>> subscribers = new ArrayList<>();

    public AutoCloseable subscribe(Consumer<T> listener) {
        subscribers.add(listener);
        return () -> subscribers.remove(listener);
    }

    public void publish(T event) {
        for (var s : List.copyOf(subscribers)) s.accept(event);
    }
}
```

# Demo.main

```java
package com.buildwithwaqar.patterns.gof.behavioral.observer;
import java.util.ArrayList;
public final class Demo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Observer walkthrough ===");
    var bus = new EventBus<String>();
    var audit = new ArrayList<String>();
    var ui = new ArrayList<String>();
    try (var s1 = bus.subscribe(e -> audit.add("audit:" + e));
         var s2 = bus.subscribe(e -> ui.add("ui:" + e))) {
      bus.publish("video.published");
      bus.publish("video.transcoded");
    }
    bus.publish("ignored-after-close");
    System.out.println("audit=" + audit);
    System.out.println("ui=" + ui);
  }
}
```

# Walkthrough Output

```
=== Observer walkthrough ===
audit=[audit:video.published, audit:video.transcoded]
ui=[ui:video.published, ui:video.transcoded]
```

# Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

# Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Observer
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

# Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

# Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

# Recap

- Force → naive pain → Observer → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.